

SIMATS ENGINEERING
ASSESSMENT TOOL 2
Case study

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO3: Analyze system requirements using object-oriented techniques to identify classes, attributes, and relationships and develop use case models. (BL4)

Assessment Tool Weightage: 40%

Code of Conduct:

I K.DIVYA(192511316) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:divya

Case study

QUESTIONS: Any 4 Total Marks:40

1. Online Hotel Reservation System

A travel company plans to develop an online hotel reservation platform where customers can search hotels, book rooms, make payments, and provide reviews. Hotel managers can update room availability and pricing, while administrators manage users and transactions.

Question:

Analyze the system and explain how object-oriented principles (encapsulation, inheritance, polymorphism) can be applied. Also, suggest a suitable SDLC model and justify your choice for this system.

Introduction

The Online Hotel Reservation System allows customers to search hotels, check room availability, reserve rooms, make payments and provide reviews. Hotel managers maintain room details, availability and pricing. Administrators manage users and transactions.

Application of Object-Oriented Principles

1. Encapsulation

Encapsulation means combining data and methods within a class and restricting direct access to internal data.

For example, the Room class can contain the following attributes:

- roomNumber
- roomType
- price
- availability

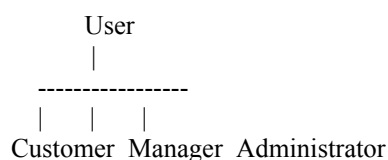
Methods can include:

- checkAvailability()
- updatePrice()
- updateAvailability()

The room details are kept private and accessed through appropriate methods.

2. Inheritance

Inheritance allows a new class to reuse the properties and methods of an existing class.



Customer, HotelManager and Administrator can inherit common attributes such as userId, name and email from the User class.

3. Polymorphism

Polymorphism allows the same method to behave differently for different objects.

For example:

`makePayment()`

can have different implementations for:

- `CreditCardPayment`
- `DebitCardPayment`
- `UPIPayment`

Thus, the same operation can support different payment methods.

Major Classes

- `User`
- `Customer`
- `Hotel`
- `Room`
- `Reservation`
- `Payment`
- `Review`
- `HotelManager`
- `Administrator`

Suitable SDLC Model – Agile Model

The Agile SDLC model is suitable for this system because requirements can change frequently.

Justification

1. Features can be developed in small iterations.
2. Customer feedback can be obtained continuously.
3. Payment and booking modules can be tested separately.
4. Changes in room pricing or booking functionality can be incorporated easily.
5. Continuous testing improves system reliability.
6. Working software can be delivered early.

Conclusion

Object-oriented principles provide reusable, secure and flexible software design. Agile development is suitable because the hotel reservation system requires continuous feedback and frequent feature updates.

2. University Management System

A university intends to automate its operations including student admissions, course registration, examination management, and result processing. Different users such as students, faculty members, examination staff, and administrators interact with the system.

Question:

Design a structured approach using object-oriented concepts to model this system. Identify key classes and relationships, and recommend an SDLC model for efficient development with reasoning.

Introduction

The University Management System automates student admission, course registration, examination management and result processing. Students, faculty members, examination staff and administrators interact with the system.

Major Classes

Student

Attributes:

- studentId
- name
- department
- email

Methods:

- applyAdmission()
- registerCourse()
- viewResult()

Faculty

Attributes:

- facultyId
- name
- department

Methods:

- manageCourse()
- enterMarks()
- updateAttendance()

Course

Attributes:

- courseId
- courseName
- credits

Methods:

- addCourse()
- updateCourse()

Examination

Attributes:

- examId
- examName
- date

Methods:

- scheduleExam()
- conductExam()

Result

Attributes:

- resultId
- marks
- grade

Methods:

- calculateGrade()
- publishResult()

Administrator

Attributes:

- adminId
- name

Methods:

- manageUsers()
- manageCourses()
- generateReports()

Relationships

Student — registers — Course

Faculty — teaches — Course

Student — attends — Examination

Student — receives — Result

Administrator — manages — University System

A student can register for multiple courses, and a course can have multiple students. Therefore, the relationship between Student and Course is many-to-many.

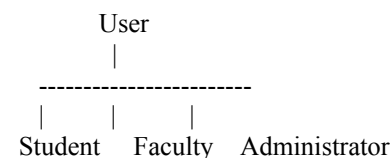
Object-Oriented Approach

Encapsulation

Student information, marks and examination details are stored inside their respective classes and accessed through methods.

Inheritance

A general User class can be created.



Polymorphism

Different users can perform a common operation such as login() with different access permissions.

Suitable SDLC Model – Agile

Agile is suitable because the university may continuously introduce changes in:

- Course registration
- Examination schedules
- Result processing
- Admission procedures
- User requirements

The system can be developed module by module and tested after every iteration.

Conclusion

An object-oriented design provides modularity, reusability and maintainability. Agile development allows the university to continuously improve the system based on user feedback.

3. Online Retail Shopping Platform

A retail company wants to develop an e-commerce platform where customers can browse products, place orders, track deliveries, and make online payments. Vendors manage product catalogs, while administrators monitor transactions and inventory.

Question:

Discuss how modular software design can be achieved using object-oriented techniques. Evaluate different SDLC models and select the most appropriate one for this scenario with justification.

Introduction

The Online Retail Shopping Platform allows customers to browse products, place orders, make payments and track deliveries. Vendors manage product catalogs, while administrators monitor transactions and inventory.

Modular Object-Oriented Design

The system can be divided into the following independent modules:

1. User Management
2. Product Management
3. Shopping Cart
4. Order Management
5. Payment Management
6. Delivery Management
7. Inventory Management
8. Administration

Each module can contain its own classes and can be developed and maintained independently.

Major Classes

Customer

Attributes:

- customerId
- name
- email

Methods:

- browseProducts()
- placeOrder()
- trackOrder()

Product

Attributes:

- productId
- name
- price
- quantity

Methods:

- displayProduct()
- updatePrice()

Cart

Attributes:

- cartId
- items

Methods:

- addProduct()
- removeProduct()
- calculateTotal()

Order

Attributes:

- orderId
- orderDate
- status

Methods:

- createOrder()
- cancelOrder()
- trackOrder()

Payment

Attributes:

- paymentId
- amount
- status

Methods:

- makePayment()
- verifyPayment()

Vendor

Attributes:

- vendorId
- name

Methods:

- addProduct()
- updateProduct()
- manageInventory()

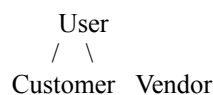
Object-Oriented Techniques

Encapsulation

Product price, customer information and payment information are protected within their respective classes. They are accessed through methods.

Inheritance

A common User class can be inherited by different types of users.



Administrator can also inherit common user properties.

Polymorphism

Different payment methods can implement the same method:

makePayment()

For example:

CreditCardPayment
 UPIPayment
 NetBankingPayment

Each class can provide its own implementation of makePayment().

Evaluation of SDLC Models

Waterfall Model

Waterfall provides a structured development process but is less flexible when requirements change.

Spiral Model

Spiral provides strong risk analysis but may require more time and resources.

Agile Model

Agile provides iterative development, continuous testing and customer feedback.

Selected Model – Agile

Agile is the most suitable model for this scenario because e-commerce requirements change frequently.

Justification

1. Frequent feature updates are possible.
2. Customer feedback can be incorporated.
3. Payment and security modules can be tested continuously.
4. New products and vendors can be added easily.
5. Different teams can work on different modules.
6. Working software can be released in iterations.

Conclusion

Object-oriented modular design makes the shopping platform reusable, maintainable and scalable. Agile is the most appropriate SDLC model because the system requires continuous improvement and frequent updates.

4. Smart Waste Management System

A municipality introduces a smart waste management system where IoT-enabled bins monitor waste levels and automatically notify collection vehicles when bins are full. Citizens can also report waste-related issues through a mobile application.

Question:

Apply object-oriented system development principles to design this system. Identify the life cycle model best suited for handling real-time data, sensor integration, and evolving requirements, and explain why.

Introduction

The Smart Waste Management System uses IoT-enabled bins to monitor waste levels. When a bin becomes full, the system automatically sends a notification to collection vehicles. Citizens can also report waste-related issues using a mobile application.

Major Classes

SmartBin

Attributes:

- binId
- location
- wasteLevel
- status

Methods:

- measureWasteLevel()
- sendNotification()
- updateStatus()

Sensor

Attributes:

- sensorId
- sensorType
- reading

Methods:

- collectData()
- sendData()

Collection Vehicle

Attributes:

- vehicleId
- driverName
- location

Methods:

- receiveNotification()
- collectWaste()

Citizen

Attributes:

- citizenId
- name
- contact

Methods:

- reportIssue()
- viewStatus()

WasteReport

Attributes:

- reportId
- location
- description
- status

Methods:

- createReport()
- updateReport()

ManagementSystem

Attributes:

- systemId

Methods:

- monitorBins()
- assignVehicle()
- generateReport()

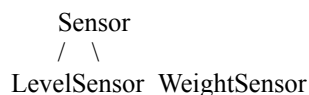
Object-Oriented Development

Encapsulation

The waste level and sensor readings are maintained inside the SmartBin and Sensor classes. This protects the internal data and allows it to be accessed through suitable methods.

Inheritance

Different sensor types can inherit from a common Sensor class.



Similarly, different users can inherit from a common User class.

Polymorphism

Different sensors can implement:

collectData()

in different ways depending on the sensor type.

For example, a LevelSensor may measure the amount of waste, while a WeightSensor measures the weight of waste.

Suitable SDLC Model – Spiral Model

The Spiral Model is suitable because this system contains hardware, IoT sensors, real-time data and software components.

Justification

1. Risk analysis is performed in every iteration.
2. Sensor integration can be tested progressively.
3. Real-time communication can be validated at each stage.
4. Hardware and software risks can be identified early.
5. Requirements can evolve during development.
6. Prototypes can be developed before complete implementation.

Conclusion

Object-oriented design provides modularity and reusability for IoT components. The Spiral Model is appropriate because the project contains technological and integration risks and requires continuous refinement.

RUBRICS FOR CALCULATION:

Case study

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding ; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding ; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand